



SUPABASE + VERCEL

Why this stack — a walkthrough using Client Pulse

INTERNAL LEARNING SESSION • ~60-75 MIN

Today's Session



Total run time: ~60 minutes (push hands-on to a follow-up if discussion runs long)

What We're Building — and the Question We're Answering

CP

Client Pulse

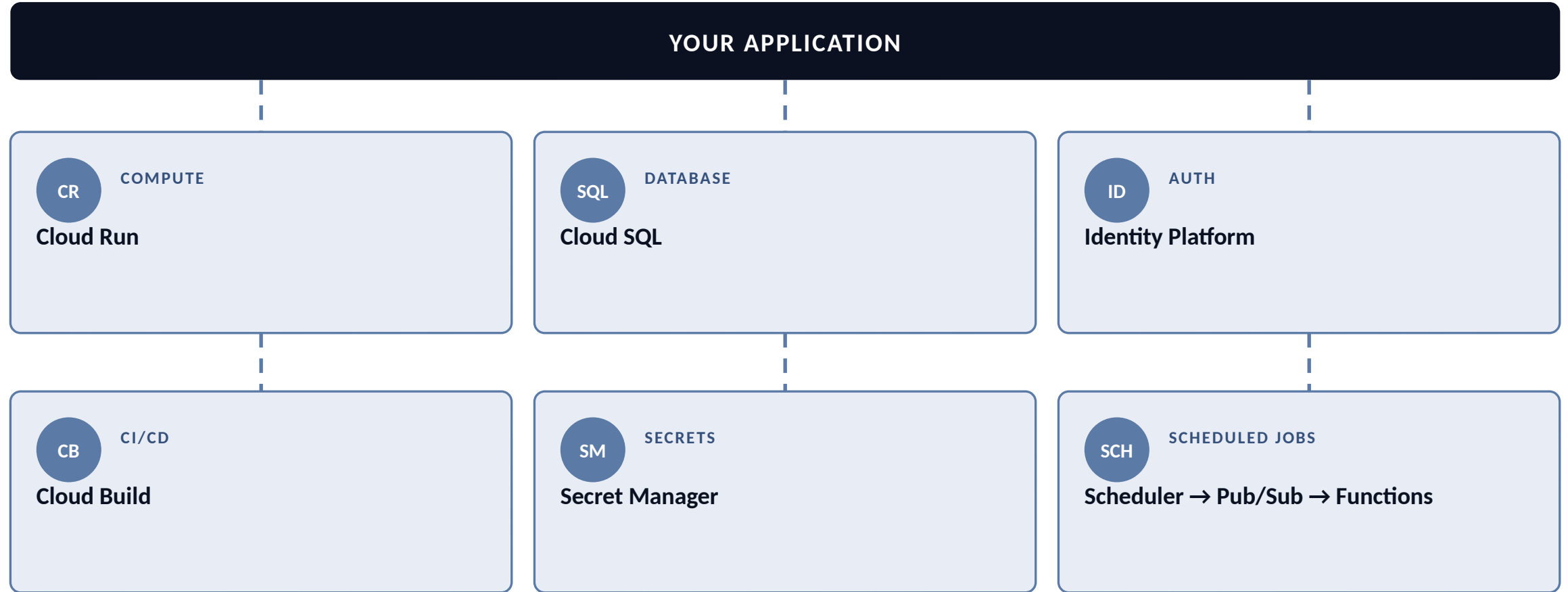
- A per-client AI reporting platform for Sales & Customer Success
- Scheduled, per-account health reports — not a generic monthly digest
- Pulls together calls, tickets, NPS, revenue & expansion signal
- Touches nearly every Supabase + Vercel primitive: DB, auth, storage, scheduled jobs, AI calls, deploys

THE CORE QUESTION

“Why is Supabase + Vercel a legitimate choice here — versus the GCP-style stack we're used to?”

Everything else in this session is in service of answering that — concretely, using this project as the example.

The GCP-Style Architecture for This App



6 separate products, each with its own IAM surface — wired together by you.

One Project, One Postgres Database

SUPABASE PROJECT

everything below is layered on top of it

PostgreSQL Database

AU

Auth

auth.users sits in the same DB
— auth.uid() is usable directly
inside policies.

ST

Storage

Files & generated reports —
the same RLS model applies to
buckets.

EF

Edge Functions

Deno serverless functions
colocated with your data, no
separate compute product.

CR

pg_cron

Scheduled jobs run inside
Postgres — one SQL statement,
not three products.

RT

Realtime

Subscribe to table changes live,
e.g. report-generation status.

Same product. Same policies. One IAM surface.

Access Control Lives in the Database



App-Layer Checks (GCP-style)

- Every endpoint must remember to filter by user
- Easy to forget on a new route or background job
- Enforcement lives in application code, not the data
- One missed check anywhere = a data leak

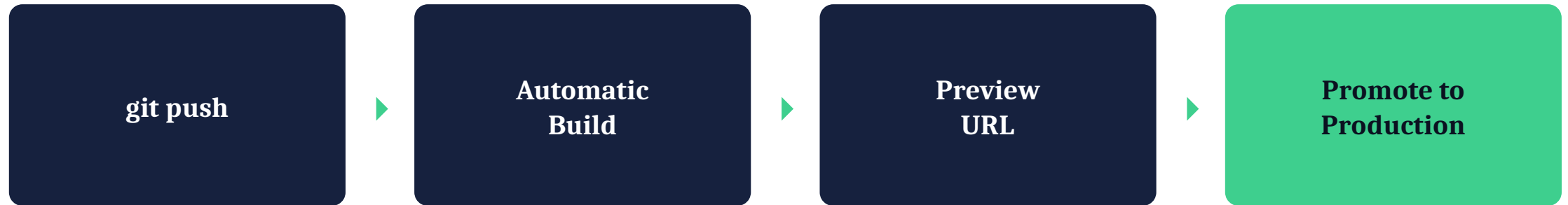


RLS Policy (Postgres)

```
create policy "reps see own clients"
  on reports for select
  using (rep_id = auth.uid());
```

Written once. Enforced by Postgres itself — no matter which endpoint, script, or tool runs the query.

From Git Push to Live URL



Preview Deployments

Every branch / PR gets its own URL automatically — no manual traffic config.



Env Vars per Environment

Production / Preview / Development scoped variables, set once in the dashboard.



Cron Jobs

Native scheduled triggers — for our monthly / quarterly report generation.

Head to Head

Concern	GCP-Style Stack	Supabase + Vercel
DB + Auth + Storage	3 separate products, reconciled in app code	1 Postgres project — RLS is the shared enforcement layer
Access control	App-layer checks, easy to bypass via a forgotten endpoint	RLS policy, enforced at the DB no matter the entry point
Scheduled job	Scheduler + Pub/Sub + Function (3 IAM bindings)	1 SQL cron.schedule(...), or 1 Vercel cron config
Deploy pipeline	Dockerfile, Cloud Build, Artifact Registry	git push — done
Preview environments	Manual traffic configuration	Automatic, per branch
Ops burden	You: patching, connection pooling, backups	Managed
Where it stops making sense	—	Extreme scale, deep GCP-ecosystem lock-in (BigQuery, Dataflow)

The Honest Trade-off



What You Give Up

- Raw infrastructure control
- Deep integration with an existing GCP ecosystem (BigQuery, Dataflow)
- Fine-grained customization for multi-region, write-heavy workloads



What You Gain

- Minutes to first deploy, not days
- One IAM surface instead of five
- No ops burden for DB patching, pooling, or backups
- Preview environments for free, on every branch

Right trade for an internal reporting tool with a defined set of users. A different conversation at extreme scale.

Let's Stand Up the Skeleton



The smallest possible slice — just enough to touch every layer once before we build the full Client Pulse platform.

NEXT UP

Build Client Pulse

The full per-account reporting platform, using everything from today.

1

Share the reference screenshot

so the generated reports match our existing visual style.

2

Use mock data first

synthetic clients, calls, tickets & NPS — no real customer data touched.

3

Scaffold the smallest slice

one client, one manually-triggered report, before adding scheduling.